

Simulating Asset Prices using Conditional Time-Series GAN

Riasat Ali Istiaque, Chi Seng Pun, Song Yuli

School of Physical and Mathematical Sciences, Nanyang Technological University, Singapore

Introduction

To generate realistic time-series data using a data-driven model

In finance, practical applications often require out-of-sample time-series data, simulated under specific conditions

- Build on TS-GAN's [1] five-network architecture
- Employ the moving window approach
- Amend the loss functions for feature and latent spaces
- Generate out-of-sample, conditional, multivariate data
- Corroborate empirically observed stylized facts
- Explore a potential application in risk management

Model: CTS-GAN

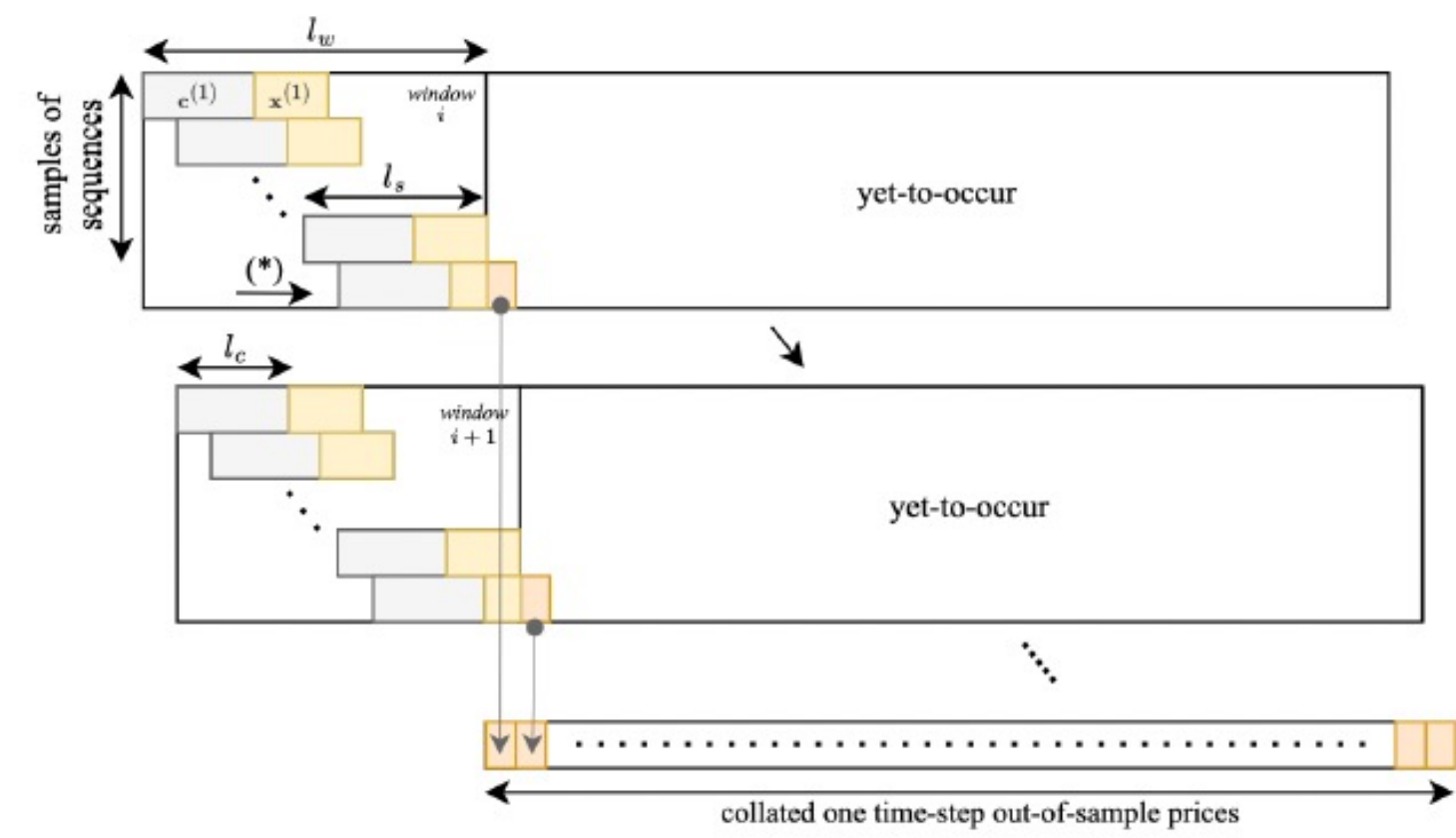


Figure 1: The moving window approach is applied to train CTS-GAN across all windows. (*) shows the post-training procedure of using a one-day offset of the conditional data from the final sequence in order to generate data whose final value would be one time-step outside of the window. This simulated data is collated and forms the out-of-sample generated data.

Training

$$\min_G \max_D \mathbb{E}_{\mathbf{x} \sim \mathbb{P}_X} [\log D(\mathbf{x}; \mathbf{c})] + \mathbb{E}_{\mathbf{z} \sim \mathbb{P}_Z} [\log (1 - D(G(\mathbf{z}; \mathbf{c}); \mathbf{c}))]$$

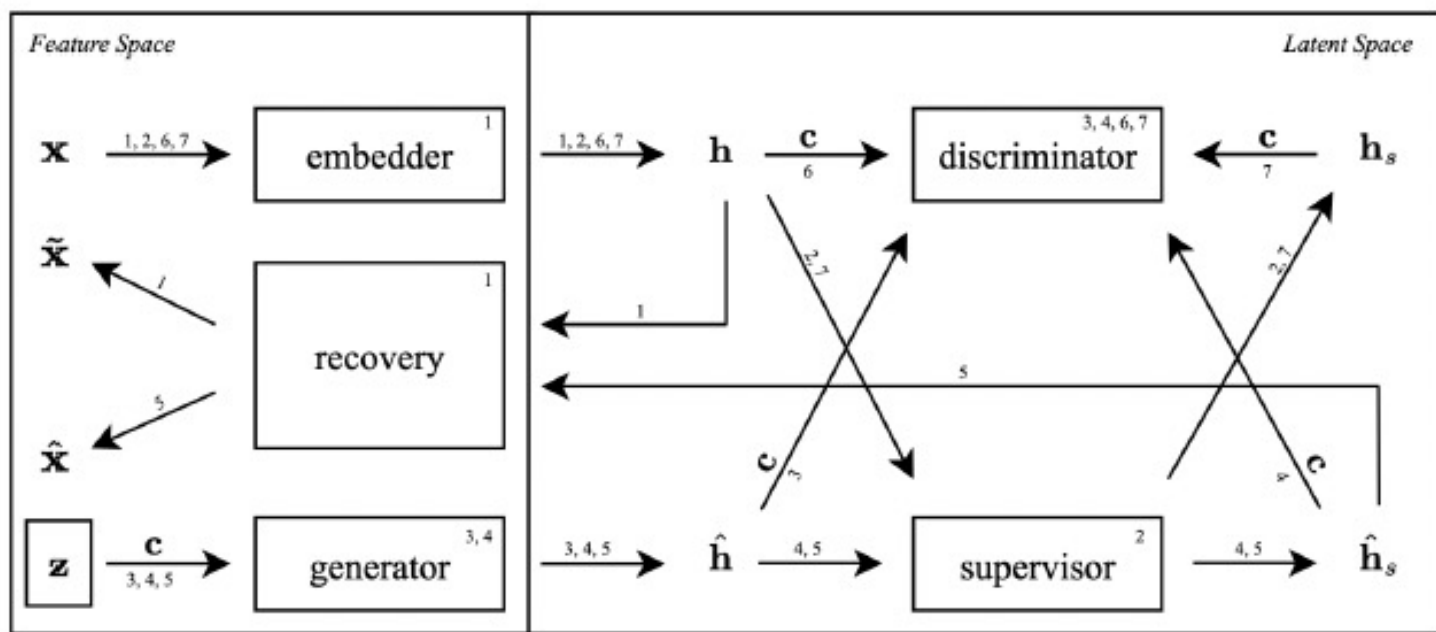


Figure 2: The summarized training scheme of CTS-GAN. The arrows indicate the flow of data through the various networks. The numbers on the arrows indicate the label of the flow, while the numbers at the corner of the networks, indicate under which flow that particular network weights are trained.

Implementation

```

1 input: window length  $l_w$ , number of iterations  $n_i$ , batch size  $m$ , sequence length  $l_s$ , number of generator iterations per discriminator iteration  $n_g$ , condition length  $l_c$ , learning rates for all five networks  $\ell_R, \ell_T, \ell_G, \ell_A$  and  $\ell_D$  and regularization parameters  $\beta, \gamma, \eta$  and  $\lambda$ 
2 output: weights for the recovery  $\theta_R$ , supervisor  $\theta_S$  and generator  $\theta_G$  networks across all windows
3 begin
4   compute number of windows  $n_w$  using  $l_w$ ;
5   foreach  $w \in \{1, \dots, n_w\}$  do
6     if  $w = 1$  then
7       initialize  $\theta_E, \theta_R, \theta_S, \theta_G$  and  $\theta_D$ ;
8     else
9       load  $\theta_E, \theta_R, \theta_S, \theta_G$  and  $\theta_D$  for  $w - 1$ ;
10    for  $n_i$  iterations do
11      get sequences  $\{\mathbf{p}^{(j)}\}_{j=1}^m$  of length  $l_s$ ;
12      filter sequences  $\{\mathbf{x}^{(j)}\}_{j=1}^m$  of length  $l_s - l_c$ ;
13      1 compute  $\mathcal{L}_R$ ;
14        update  $\theta_E$  and  $\theta_R$  using  $\ell_R$ ;
15      for  $n_i$  iterations do
16        get sequences  $\{\mathbf{p}^{(j)}\}_{j=1}^m$  of length  $l_s$ ;
17        filter sequences  $\{\mathbf{x}^{(j)}\}_{j=1}^m$  of length  $l_s - l_c$ ;
18      2 compute  $\mathcal{L}_T$ ;
19        update  $\theta_S$  using  $\ell_T$ ;
20      for  $n_i$  iterations do
21        for  $n_g$  iterations do
22          get sequences  $\{\mathbf{p}^{(j)}\}_{j=1}^m$  of length  $l_s$ ;
23          filter sequences  $\{\mathbf{c}^{(j)}\}_{j=1}^m$  of length  $l_c$ ;
24          filter sequences  $\{\mathbf{x}^{(j)}\}_{j=1}^m$  of length  $l_s - l_c$ ;
25          get sequences  $\{\mathbf{z}^{(j)}\}_{j=1}^m$  of length  $l_s - l_c$ ;
26          construct  $\{\mathbf{cz}^{(j)}\}_{j=1}^m$  of length  $l_s - l_c$ ;
27        3 compute  $\beta \mathcal{L}_U + \mathcal{L}_S + \gamma \|\tilde{\mathbf{x}} - \mathbf{x}\|_2$ ;
28          update  $\theta_G$  using  $\ell_G$ ;
29        4 compute  $\eta \mathcal{L}_R + \lambda \mathcal{L}_T$ ;
30          update  $\theta_E$  and  $\theta_R$  using  $\ell_A$ ;
31        get sequences  $\{\mathbf{p}^{(j)}\}_{j=1}^m$  of length  $l_s$ ;
32        filter sequences  $\{\mathbf{c}^{(j)}\}_{j=1}^m$  of length  $l_c$ ;
33        filter sequences  $\{\mathbf{x}^{(j)}\}_{j=1}^m$  of length  $l_s - l_c$ ;
34        get sequences  $\{\mathbf{z}^{(j)}\}_{j=1}^m$  of length  $l_s - l_c$ ;
35        construct  $\{\mathbf{cz}^{(j)}\}_{j=1}^m$  of length  $l_s - l_c$ ;
36      5 compute  $-\beta \mathcal{L}_U - \mathcal{L}_S$ ;
37        update  $\theta_D$  using  $\ell_D$ ;
38    save  $\theta_E, \theta_R, \theta_S, \theta_G$  and  $\theta_D$  for  $w$ ;

```

$$\mathcal{L}_R := \|\tilde{\mathbf{x}} - \mathbf{x}\|_2, \text{ where } \mathbf{h} = E(\mathbf{x}), \tilde{\mathbf{x}} = R(\mathbf{h})$$

$$\mathcal{L}_T := \|\mathbf{h}_s - \mathbf{h}\|_2, \text{ where } \mathbf{h}_s = S(\mathbf{h})$$

$$\mathcal{L}_U := \mathbb{E}_{\mathbf{h} \sim \mathbb{P}_H} [\log D(\mathbf{h}; \mathbf{c})] + \mathbb{E}_{\mathbf{z} \sim \mathbb{P}_Z} [\log (1 - D(G(\mathbf{z}; \mathbf{c}); \mathbf{c}))]$$

$$\mathcal{L}_S := \mathbb{E}_{\mathbf{h}_s \sim \mathbb{P}_H} [\log D(\mathbf{h}_s; \mathbf{c})] + \mathbb{E}_{\mathbf{z} \sim \mathbb{P}_Z} [\log (1 - D(S(G(\mathbf{z}; \mathbf{c})); \mathbf{c}))]$$

$$\hat{\mathbf{x}} = R(S(G(\mathbf{z}; \mathbf{c})))$$

Simulations

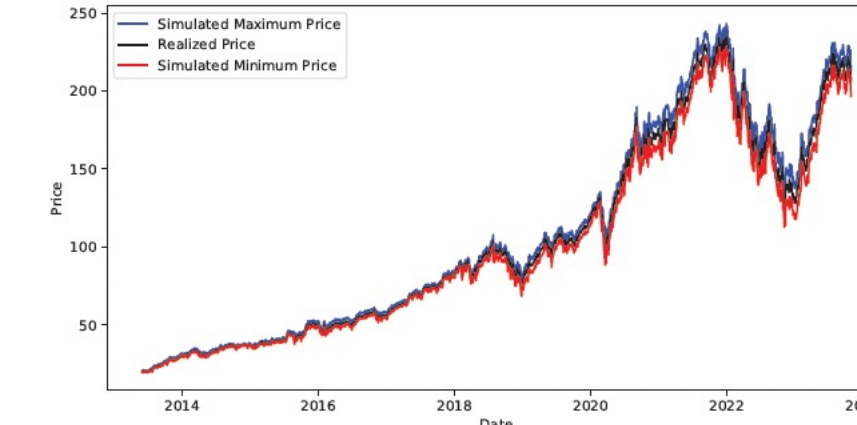


Figure 3: Maximum and minimum of the 10000 simulated prices of an EW portfolio of the five assets, across out-of-sample windows. The red line depicts the minimum achieved across 10000 simulated, EW portfolio price, while the blue line depicts the maximum achieved. The black line is the realized price across the out-of-sample prices.

Stylized Facts

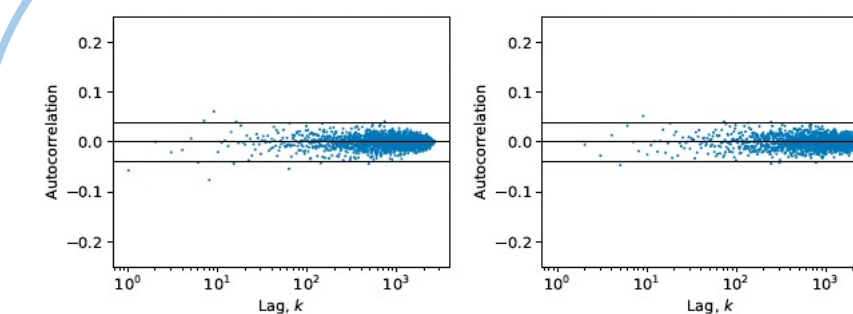


Figure 4a: Absence of Linear Autocorrelation. Out-of-sample correlograms. The left (resp. right) graph plots the realized (resp. simulated) data.

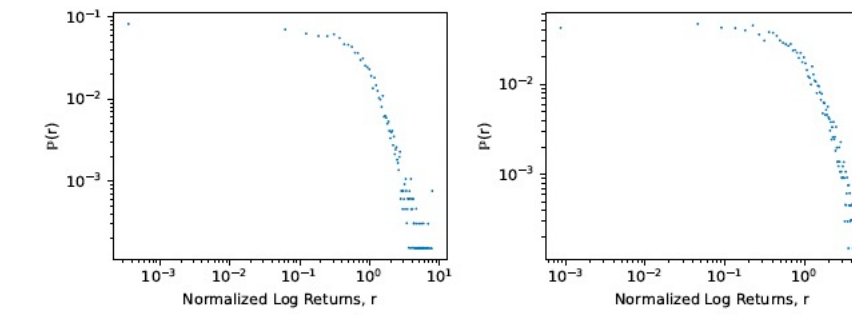


Figure 4b: Heavy-Tailed Distribution. Out-of-sample log-log plots of the density of positive normalized returns. The left (resp. right) graph plots the realized (resp. simulated) data.

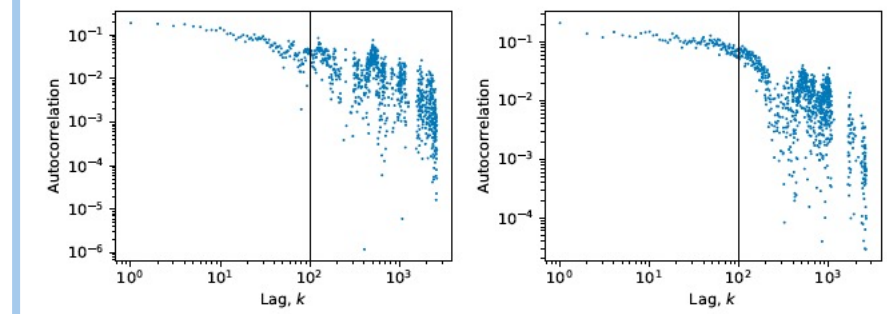


Figure 4c: Volatility Clustering. Out-of-sample log-log plots of the autocorrelation of absolute returns. The black horizontal line demarcates the time-lag $k=100$, after which the autocorrelation is reported to break down. The left (resp. right) graph plots the realized (resp. simulated) data.

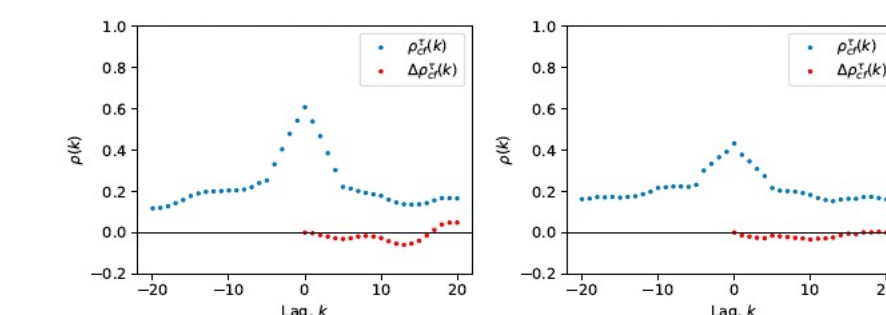


Figure 4d: Coarse-Fine Volatility Correlation. Out-of-sample plots of the correlation between course volatility and fine volatility. The blue curve is the coarse-fine volatility correlation, while the red curve is the difference between the lead coarse-fine volatility correlation and lag coarse-fine volatility correlation. $\tau=5$, representing day-week timescales. The left (resp. right) graph plots the realized (resp. simulated) data.

And 3 more

Risk Management

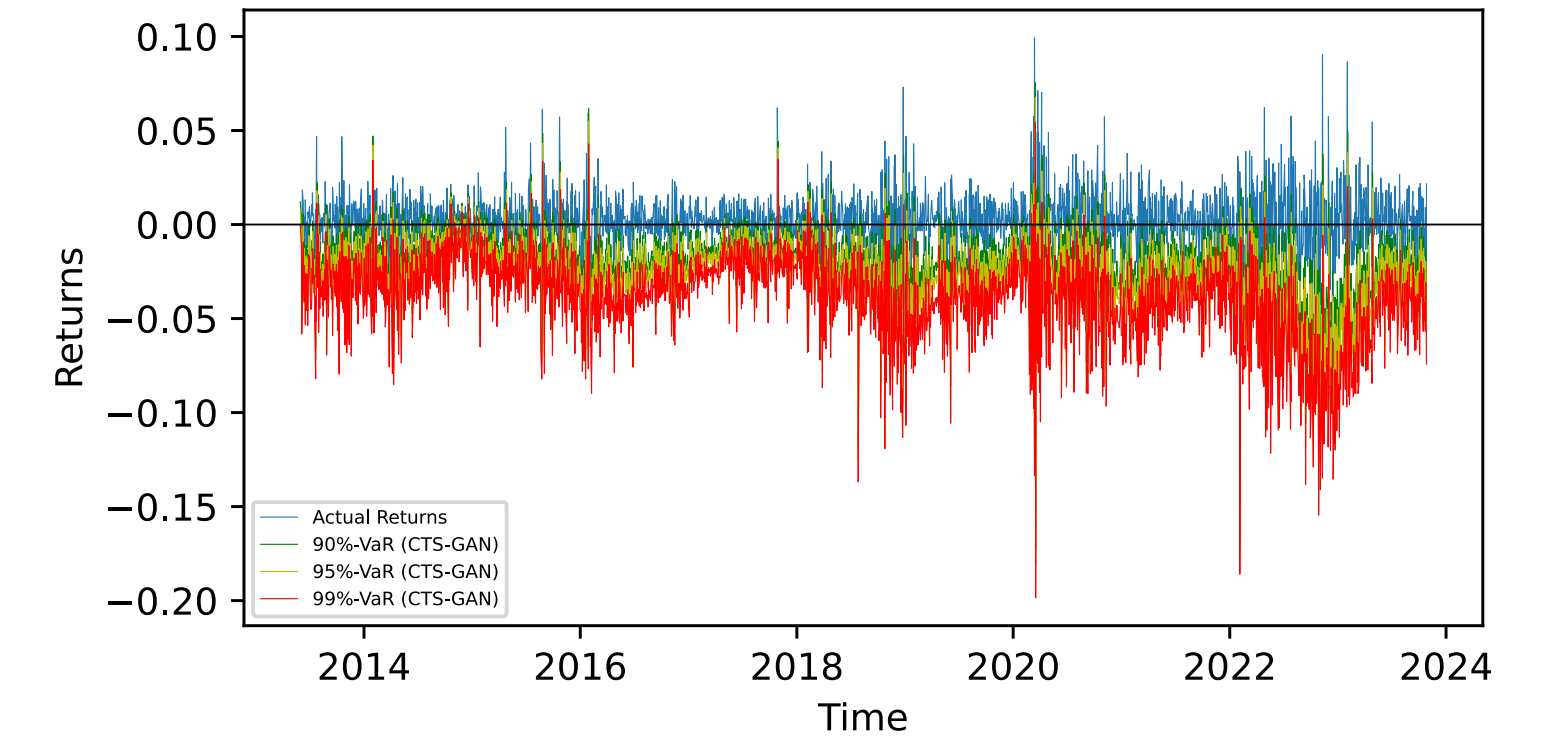
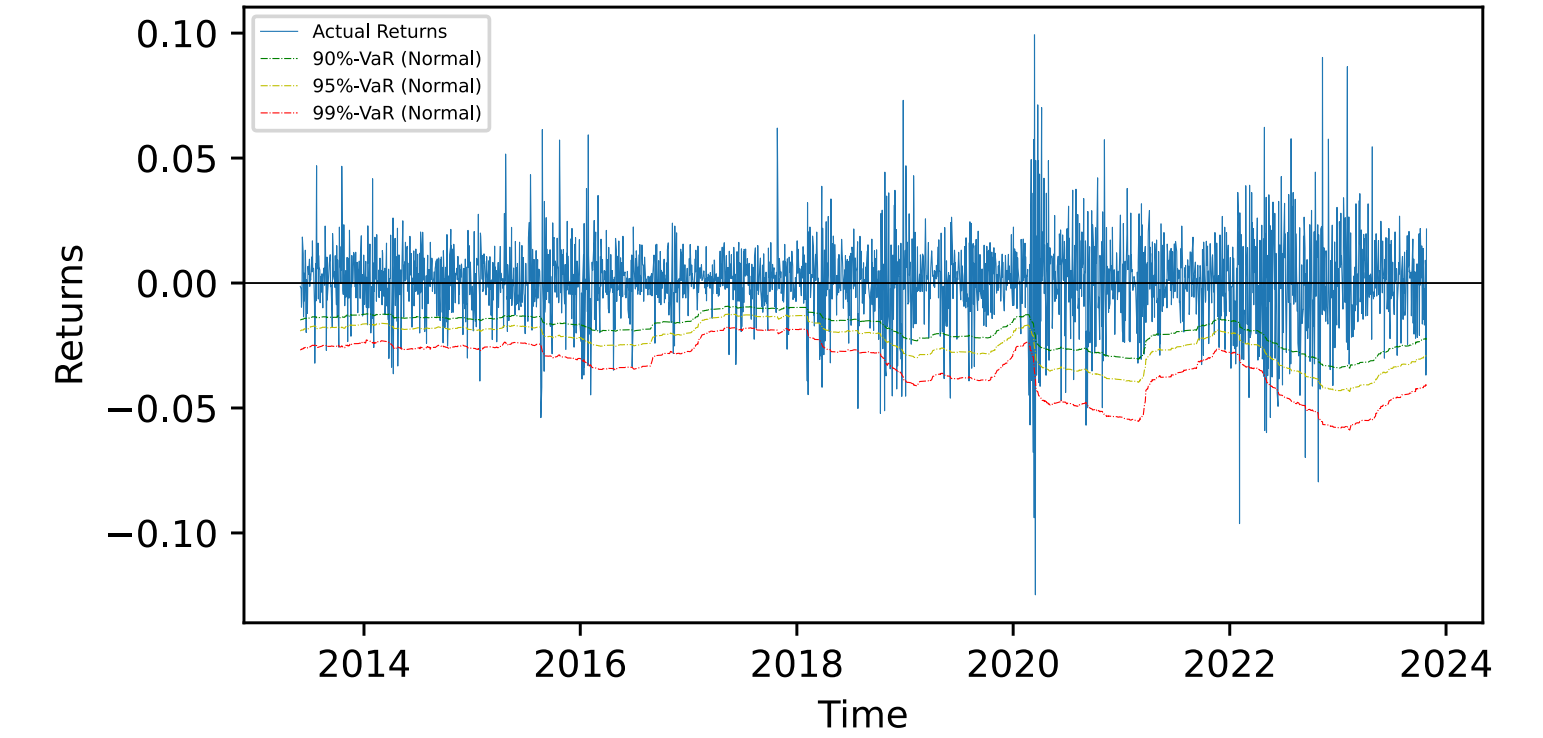


Figure 5: Out-of-sample plots of the daily VaR estimates under a multivariate normal model versus CTS-GAN. The blue curve is the realized returns. The dashed (resp. solid) curves in the top (resp. bottom) graph pertains to the daily VaR estimates at the α -confidence levels, computed from the simulated returns generated by a multivariate normal model (resp. CTS-GAN).

	Breach Prop. (%)		Breach Mag. (%)		Non-Breach Mag. (%)	
α	Normal	CTS-GAN	Normal	CTS-GAN	Normal	CTS-GAN
0.90	0.08734	0.13425	0.01257	0.01382	0.02258	0.02887
0.95	0.06064	0.08963	0.01195	0.01370	0.02754	0.03352
0.99	0.02975	0.04233	0.01153	0.01453	0.03595	0.04232

Table 1: Table of proportions, average magnitude of the breaches and average magnitude of the non-breaches, in percentage, by the realized returns against the respective estimated daily VaR at the α -confidence levels, across all the out-of-sample windows.

Conclusion

We can generate multivariate asset prices under the condition of recently observed prices. We checked that the out-of-sample prices exhibit empirically observed stylized facts and showed an application in risk management, where we compared two sets of daily VaR values, for equally-weighted portfolios.